

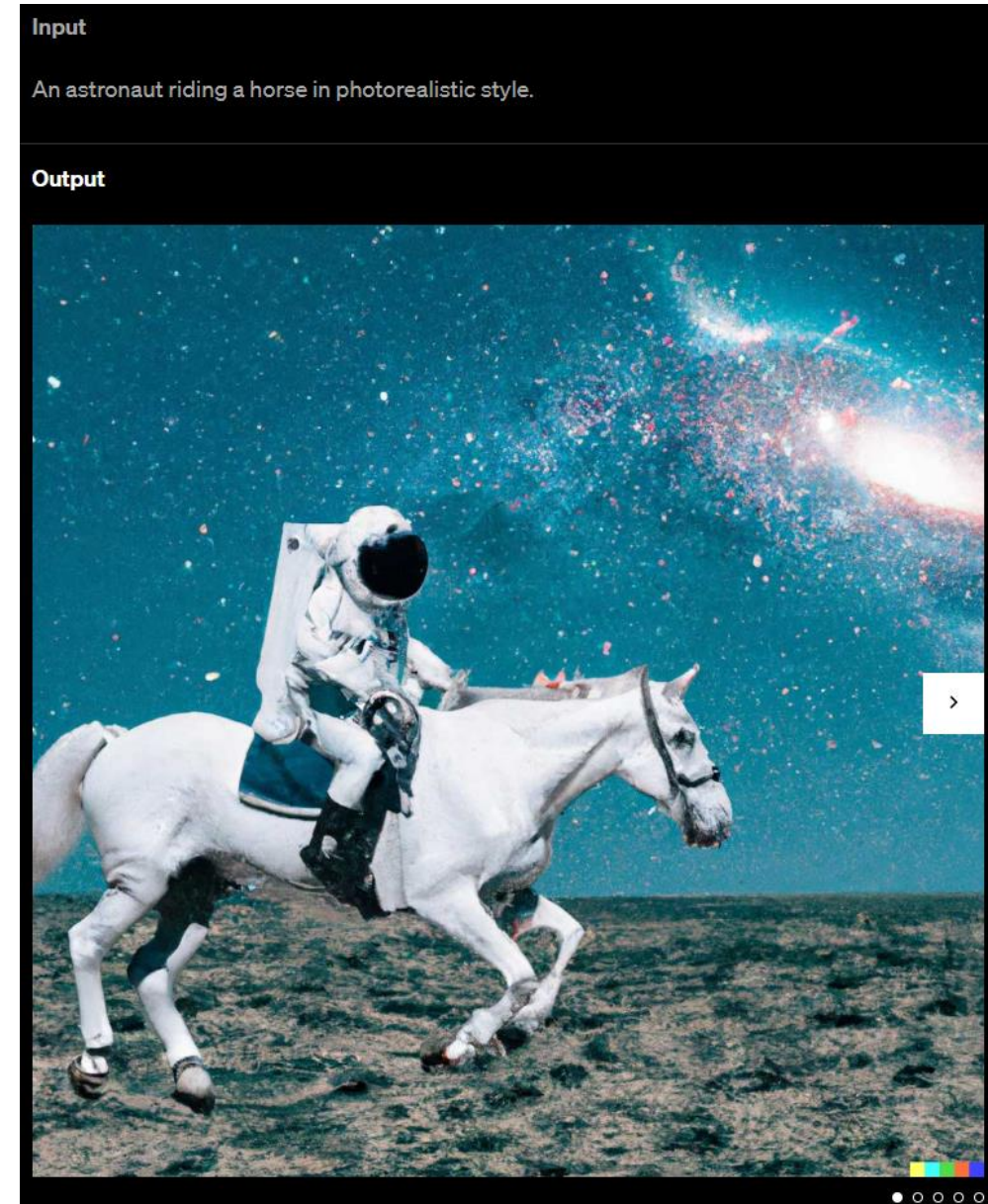
Image Synthesis

Computer Vision

idea: generate new images as variations of training data (same distribution)

usually conditioned on text (prompt)

compared to text generation, additional mechanism needed (e.g., diffusion) due to more complex image structures



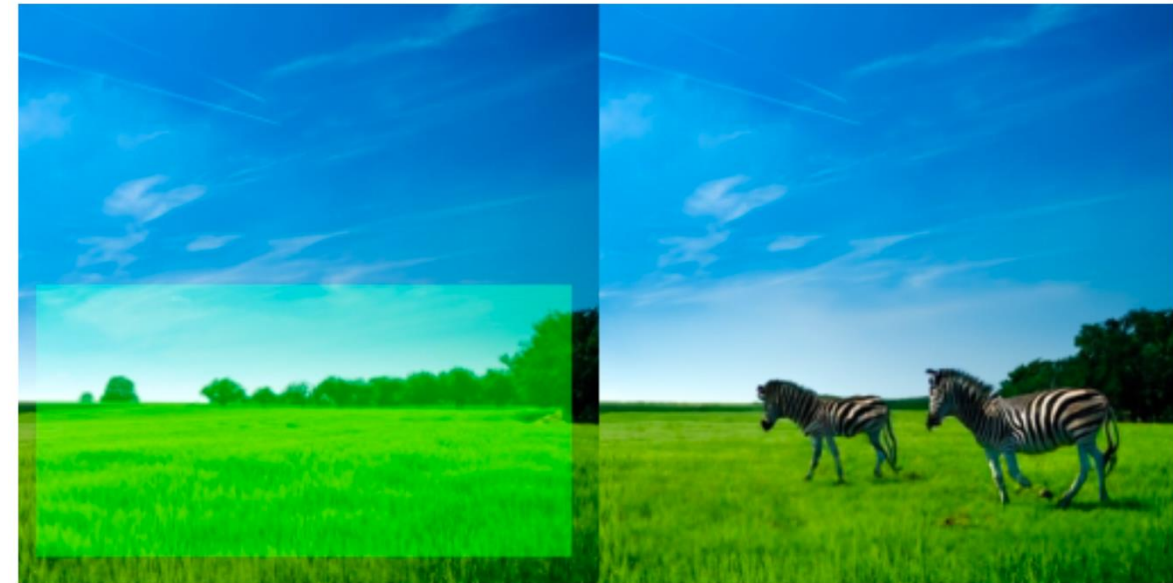
plenty of products: [DALL-E](#), [Stable Diffusion](#), [ImageGen](#), [Midjourney](#), ...

Stable Diffusion:



A scenic view of mountains at sunset, digital art

inpainting example ([GLIDE](#)):



“zebras roaming in the field”

prompt

Generative vs Predictive/Discriminative Models

discriminative models:

predict conditional probability $P(Y|\mathbf{X})$

generative models:

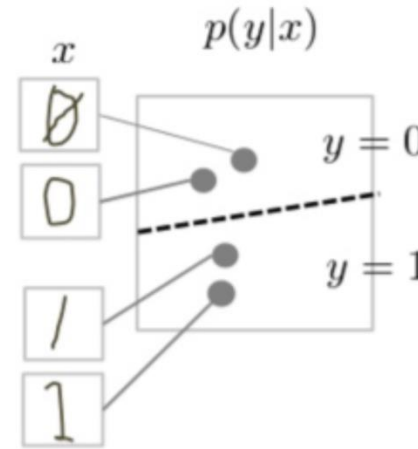
predict joint probability $P(Y, \mathbf{X})$

(or just $P(\mathbf{X}) \rightarrow$ unsupervised learning)

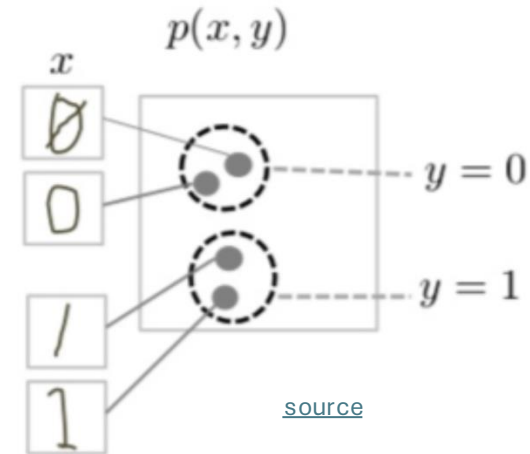
$\rightarrow$ allow to generate new data samples

Generative models can be used for predictive tasks (Bayes theorem).
But predictive models are usually better at it.

discriminative model



generative model



[source](#)

task of generative models more difficult: need to model full data distribution rather than merely find patterns in inputs to distinguish outputs

Classic ML vs Deep Learning

Generative vs Predictive Models

text generation

image synthesis

text-to-video

```
done = False
total_reward = 0
while not done:
    state = torch.tensor(state, dtype=torch.float32)
    logits = policy_network(state)
    action = torch.argmax(logits).item()
    next_state, reward, done, _ = env.step(action)
    total_reward += reward
    state = next_state
    total_rewards.append(total_reward)

# Print average reward
print("Average Reward:", np.mean(total_rewards))
```

This code uses PyTorch to implement the policy gradient method (REINFORCE algorithm) to solve the CartPole problem. The policy network is defined as a simple feedforward neural network, and the training loop updates the policy network parameters to maximize the expected reward. Finally, it evaluates the learned policy by running it for 100 episodes and prints the average reward. Adjust hyperparameters and network architecture as needed for better performance.

Can you extend this to an actor-critic method?

Certainly! Here's an extension of the previous code using the actor-critic method to solve the CartPole problem:

Message ChatGPT

ChatGPT can make mistakes. Check important info.



Prompt: Epic anime artwork of a wizard atop a mountain at night casting a cosmic spell into the dark sky that says "Stable Diffusion 3" made out of colorful energy



Prompt: A stylish woman walks down a Tokyo street filled with warm glowing neon and animated city signage. She wears a black leather jacket, a long red dress, and black boots, and carries a black purse. She wears sunglasses and red lipstick. She... +

ChatGPT

Stable Diffusion 3 — Stability AI

Sora | OpenAI

BERT family

tabular data

computer vision



Classify

Detect

Segment

Track



YOLO

Deep Learning for Generative AI

Depending on the application, there are currently two dominant approaches for generative AI:

- text generation: LLMs
- image synthesis: diffusion models

note the difference between image synthesis and multimodal understanding in LLMs
(images as additional input sequences to transformer, tokenized by splitting into patches)

Autoregressive Text Generation

LLMs (autoregressive transformers) generate one output token at a time (according to predicted probabilities)

then add it to the context for the prediction of the next token

→ resulting in a full text, sampled from predicted probabilities

(can be done in an analog way for images, with patches as tokens)

How are LLMs Generative Models then?

at each step, take the sequence of previous tokens $x_{<t}$ (context) and predict a conditional probability distribution over the next token x_t :

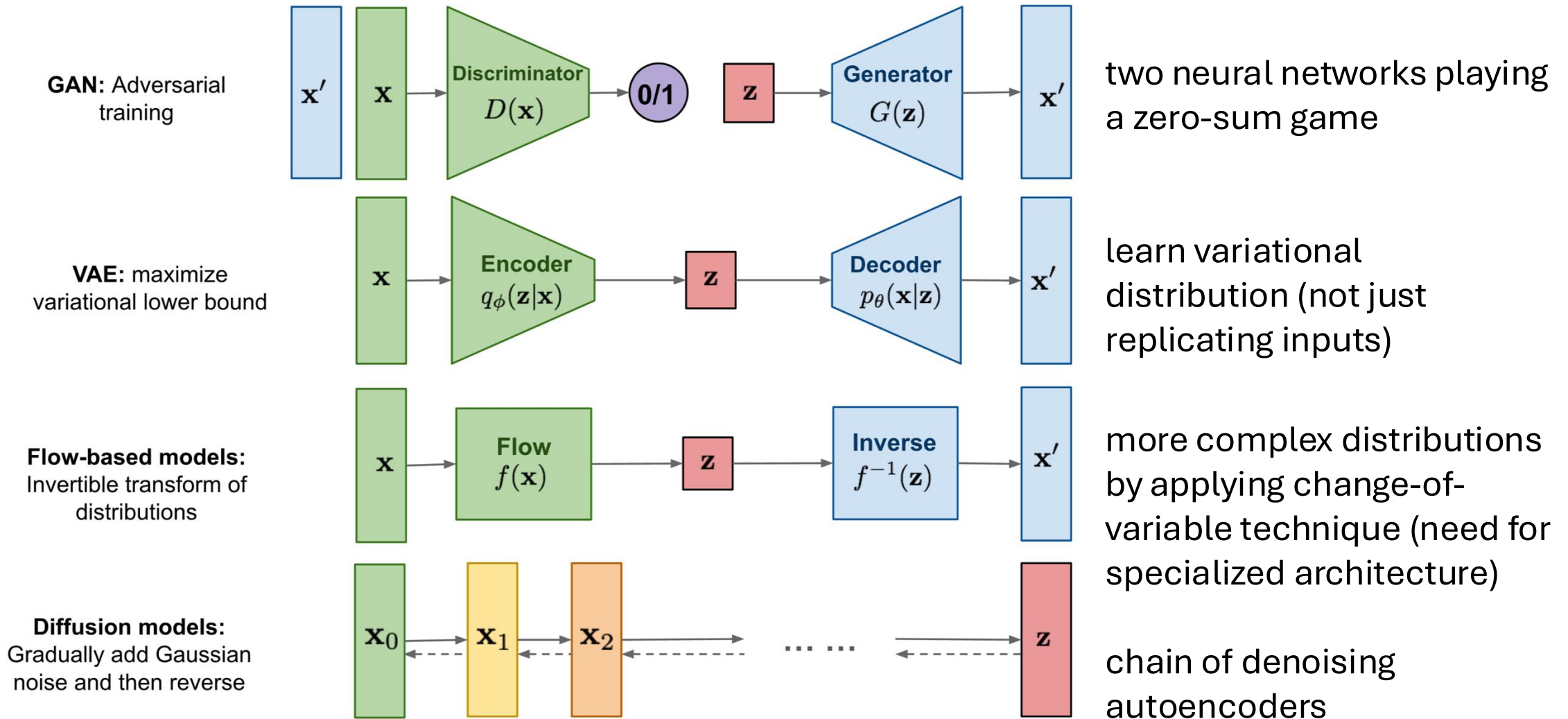
$$P(x_t | x_{<t})$$

joint probability distribution over text sequences via chain rule of probability:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t})$$

→ no direct prediction of joint probability in one shot, but implicit calculation through product of conditionals

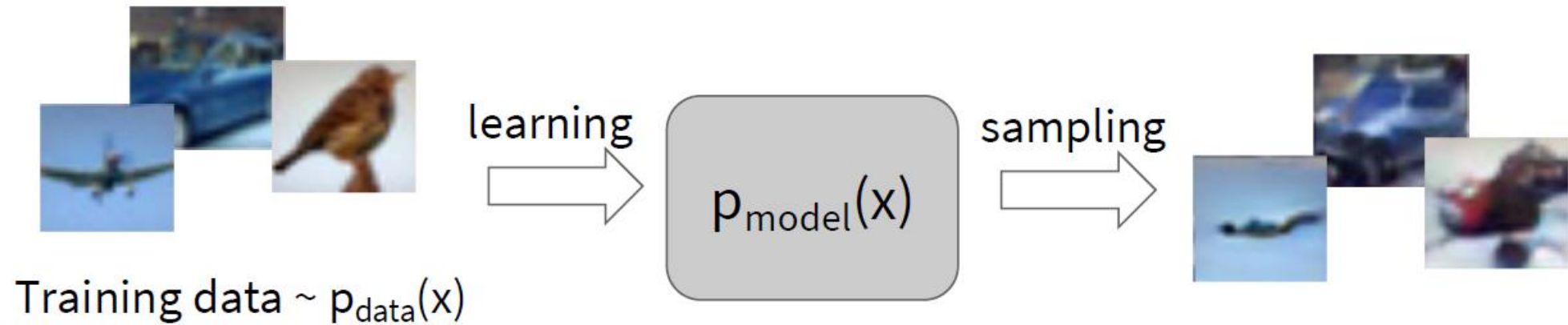
Different Model Types for Image Synthesis



[source](#)

→ generalization: [flow matching](#)

Generative Modeling



Objectives:

1. Learn $p_{\text{model}}(x)$ that approximates $p_{\text{data}}(x)$
2. Sampling new x from $p_{\text{model}}(x)$

Explicit density estimation: explicitly define and solve for $p_{\text{model}}(x)$

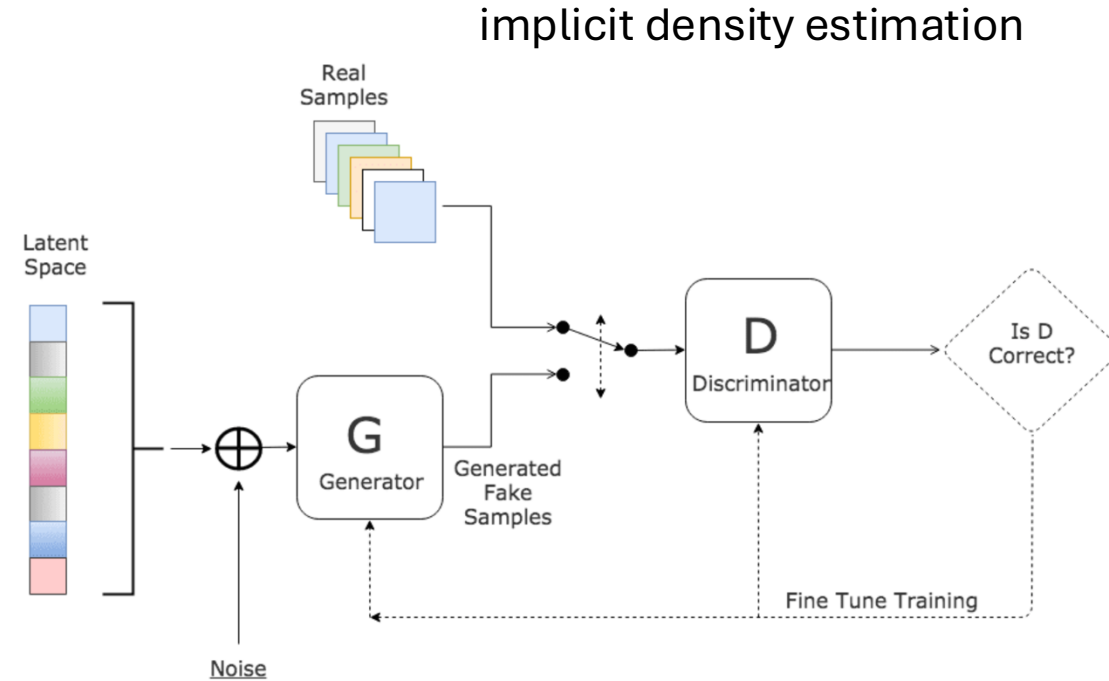
Implicit density estimation: learn model that can sample from $p_{\text{model}}(x)$ without explicitly defining it.

Generative Adversarial Networks (GAN)

two neural networks playing a zero-sum game:

- the generator network G generating new (fake) samples
- the discriminator network D trying to distinguish between real and fake samples

indirect training via D: G not trained directly to minimize reconstruction error of real samples, but to fool D → self-supervised approach



[source](#)

common loss for generator and discriminator:

$$L(x_i) = E_{x \sim p_r(x)} [\ln D(x_i)] + E_{x \sim p_g(x)} [\ln(1 - D(x_i))]$$

G trying to minimize

D trying to maximize

Conditional GANs

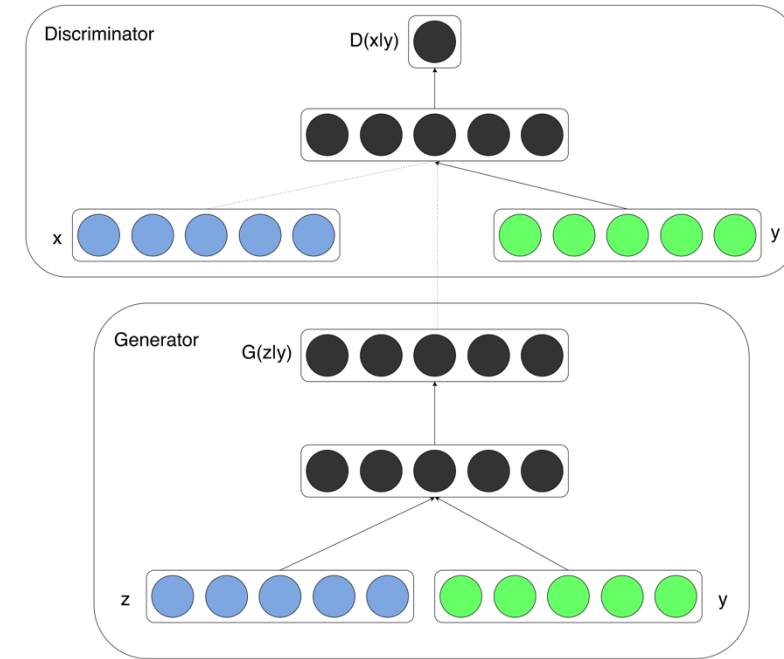
as discussed so far, generative methods give no control over what kind of data is generated (limited usability)

→ need for conditional approach (e.g., conditioning on describing text)

example GANs:

transform usual GAN to conditional model by feeding extra information y (e.g., class labels) as additional input layer into both generator and discriminator

$$L(\mathbf{x}_i) = E_{\mathbf{x} \sim p_r(\mathbf{x})} [\ln D(\mathbf{x}_i | y_i)] + E_{\mathbf{x} \sim p_g(\mathbf{x})} [\ln(1 - D(\mathbf{x}_i | y_i))]$$



[source](#)

Vector Arithmetic in GAN Latent Space



[source](#)



Variational Autoencoder (VAE)

goal: generation of variations of input data rather than compressed representation

→ learn variational distribution instead of identity function

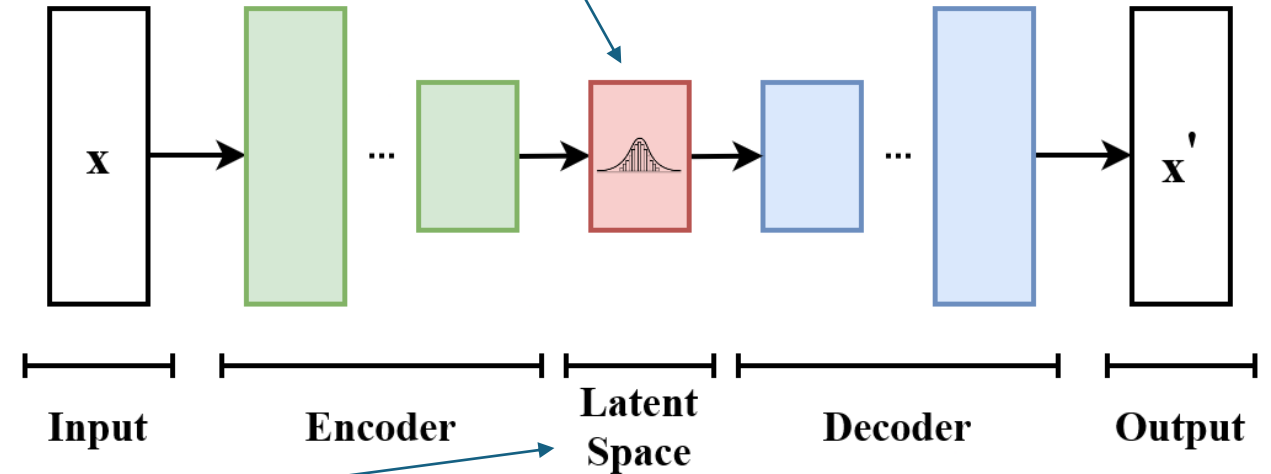
to be precise: parametrized variational distribution of latent encoding variables $\mathbf{z}$

prior (simple distribution, in usual VAE: Gaussian): $p_{\theta}(\mathbf{z})$

posterior: $p_{\theta}(\mathbf{z}|\mathbf{x}) = \frac{p_{\theta}(\mathbf{x}|\mathbf{z})p_{\theta}(\mathbf{z})}{\int p_{\theta}(\mathbf{x}|\mathbf{z})p_{\theta}(\mathbf{z})d\mathbf{z}}$

$p_{\theta}(\mathbf{x})$: mixture of Gaussians

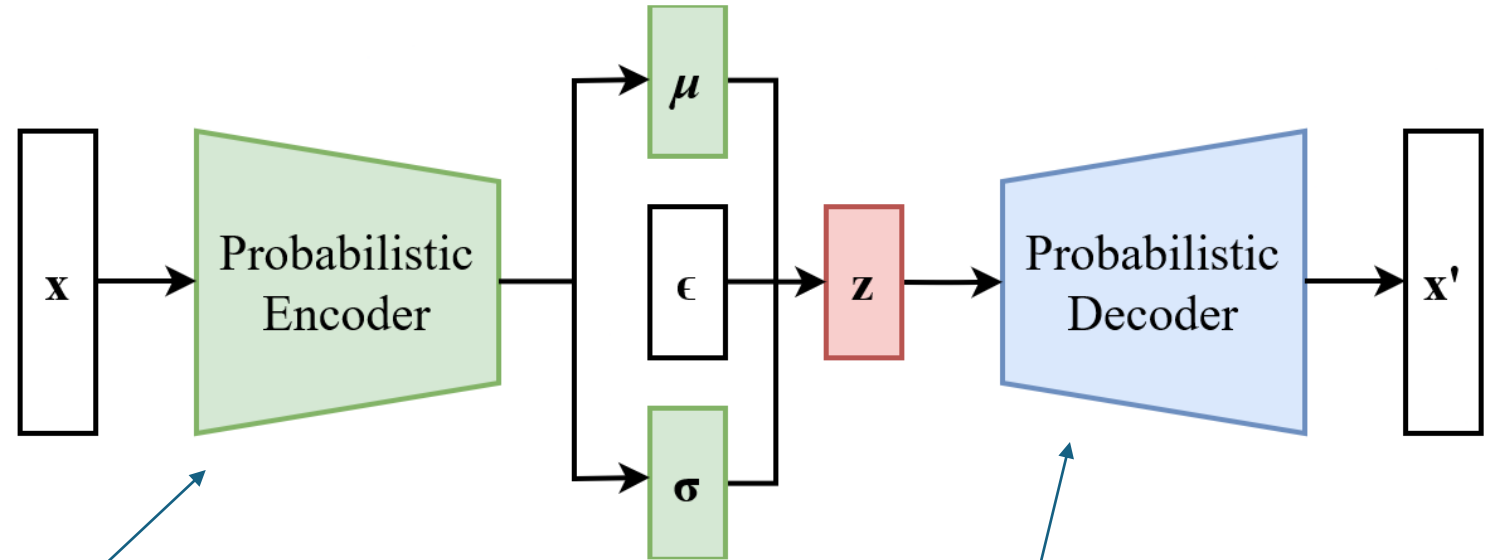
from which to sample



Variational Bayesian Method

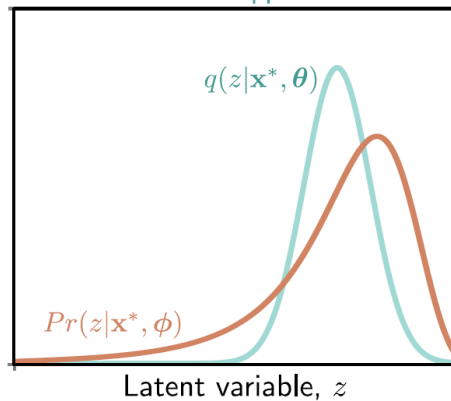
Gaussian Approximation

learn mean and variance of multivariate Gaussian with diagonal covariance structure



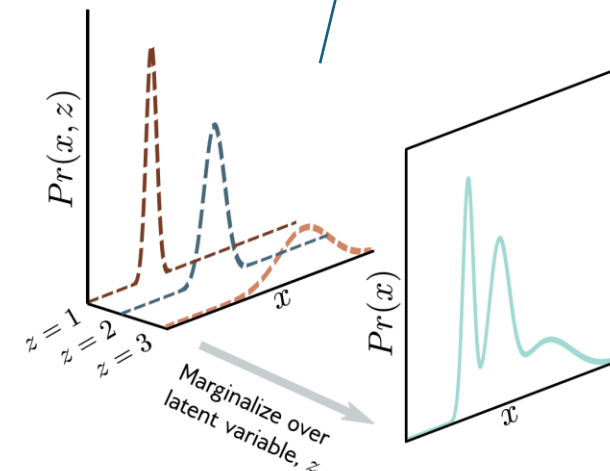
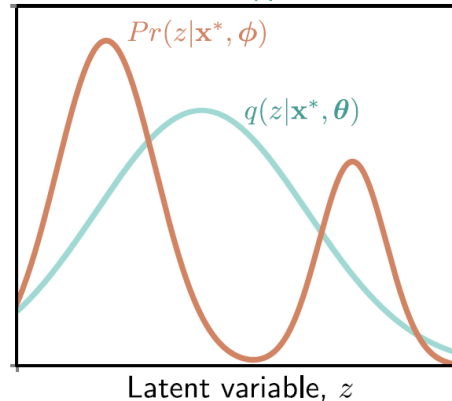
good approximation:

Posterior and approximation



poor approximation:

Posterior and approximation

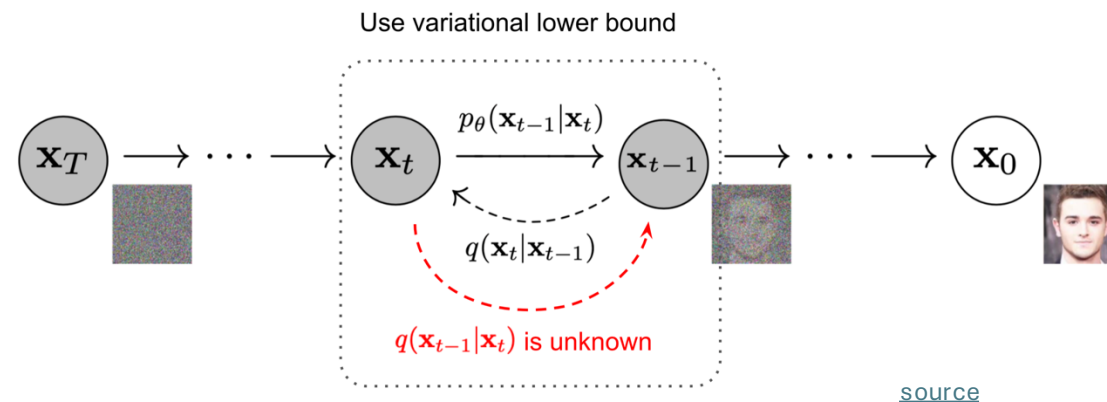


[source](#)

Diffusion

training: distort training data by successively adding random noise, then learn to reverse this process (denoising)

generation: sample random noise and run through the learned denoising process



advantages: easy to train, produce high-quality/realistic samples

can be interpreted as special case of hierarchical VAE (one latent variable generates another) with fixed encoder and latent space of same size as the data

→ more sophisticated latent space than just Gaussian mixture in VAE

Noise Prediction

Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_{\theta} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2$   
6: until converged
```

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

[source](#)

Diffusion as Chain of Denoising Autoencoders

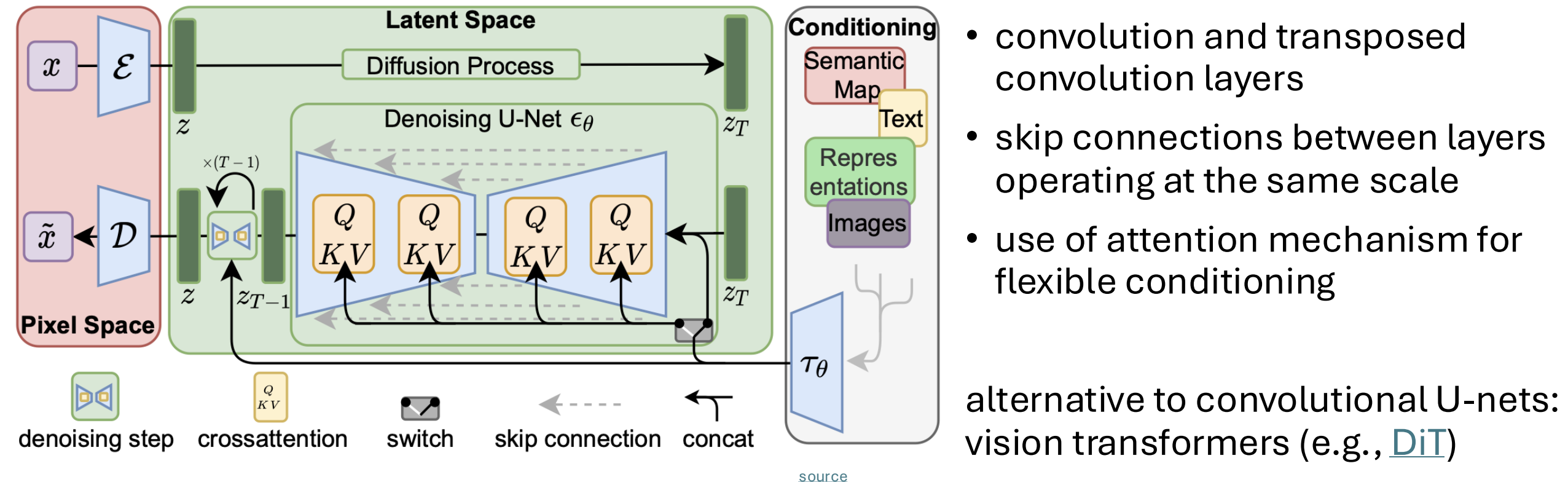
differences of diffusion models to typical denoising autoencoders:

- no bottleneck (care about output here, not internal representation): latent space with high dimensionality (same as original data)
- handle many different noise levels with single set of shared parameters

important application: [AlphaFold 3](#) uses diffusion-based architecture for protein structure prediction

Latent Diffusion

add noise to latent representation rather than raw data
→ significant speedup



- convolution and transposed convolution layers
- skip connections between layers operating at the same scale
- use of attention mechanism for flexible conditioning

alternative to convolutional U-nets:
vision transformers (e.g., [DiT](#))

Guided Diffusion

condition diffusion process on class information (label or just text)

$$\epsilon_{\theta}(\mathbf{x}_t|\emptyset) + s \cdot (\epsilon_{\theta}(\mathbf{x}_t|y) - \epsilon_{\theta}(\mathbf{x}_t|\emptyset))$$

s : hyperparameter to control tradeoff between diversity (unconditioned) and fidelity (guidance)

similar idea as softmax temperature in auto-regressive LLMs

“Pembroke Welsh corgi”



[source](#)

Outlook: Text2Anything

next step: text-to-video ([Make-A-Video](#), [Lumiere](#), [Sora](#), ...)

→ rudimentary physics understanding

at some point maybe also generation of proteins, materials, ...